

Final-FOC- ADHISH.docx

 Islington College, Nepal

Document Details

Submission ID
trn:oid::3618:109779536

Submission Date
Aug 27, 2025, 11:44 AM GMT+5:45

Download Date
Aug 27, 2025, 11:46 AM GMT+5:45

File Name
Final-FOC- ADHISH.docx

File Size
14.2 KB

9 Pages

2,104 Words

11,527 Characters



Page 1 of 13 - Cover Page

Submission ID trn:oid::3618:109779536



Page 2 of 13 - Integrity Overview

Submission ID trn:oid::3618:109779536

18% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

-  **36 Not Cited or Quoted 18%**
Matches with neither in-text citation nor quotation marks
-  **0 Missing Quotations 0%**
Matches that are still very similar to source material
-  **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 2%**  Internet sources
- 0%**  Publications
- 18%**  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Table of Contents

1. Introduction.....	5
1.1. Project Overview.....	5
1.2. Aim.....	5
1.3. Objectives.....	5
1.4. Technologies Used.....	6
2. Discussion and Analysis.....	6
2.1. Algorithm.....	7
2.2. Flowchart.....	10
2.3. Pseudocode.....	14
2.3.1. read.py.....	14
2.3.2. operations.py.....	14
2.3.3. write.py.....	18
2.3.4. main.py.....	19
2.4. Data Structure.....	21
2.4.1. Integer.....	21
2.4.2. Float.....	21
2.4.3. String.....	21
2.4.4. Boolean.....	22
2.4.5. List.....	22
2.4.6. Tuple.....	22
2.4.7. Set.....	23
2.4.8. Dictionary.....	23
3. Testing.....	24
3.1. Test 1.....	24
3.2.1 Test 2.1.....	25
3.2.2. Test 2.2.....	26
3.2.3. Test 2.3.....	27
3.2.4. Test 2.4.....	28
3.3. Test 3.....	29

3.4 Test 4.....	30
3.5.1. Test 5.1.....	32
3.5.2. Test 5.2.....	33
5. Conclusion.....	35
6. References.....	35
7. Appendix.....	36

Table of Figures

Figure 1: Flowchart	10
Figure 2: Flowchart	11
Figure 3: Flowchart	12
Figure 4: Flowchart	13
Figure 5: Integer	21
Figure 6: Float	21
Figure 7: String	21
Figure 8: Boolean	22
Figure 9: List	22
Figure 10: Tuple	22
Figure 11: Set	23
Figure 12: Dictionary	23
Figure 13: Test 1	24
Figure 14: Test 2.1	25
Figure 15: Test 2.2	26
Figure 16: Test 2.3	27
Figure 17: Test 2.4	28
Figure 18: Test 3	29
Figure 19: Test 3	30
Figure 20: Test 4	31
Figure 21: Test 4	31
Figure 22: Test 5.1	32
Figure 23: Test 5.1 Before	33
Figure 24: Test 5.1 After	33
Figure 25: Test 5.2	34
Figure 26: Test 5.2 Before	34
Figure 27: Test 5.2 After	35

Table of Tables

Table 1: Test 1	24
Table 2: Test 2.1	25
Table 3: Test 2.2	26
Table 4: Test 2.3	27
Table 5: Test 2.4	28
Table 6: Test 3	29
Table 7: Test 4	30
Table 8: Test 5.1	32
Table 9: Test 5.2	33

1. Introduction

1.1. Project Overview

The rapid digital upgrade in retail and wholesale industries has pushed many businesses to implement intelligent software and programs to complete various operations. This project, titled Shoes Wholesale System, is designed to simplify inventory and logistics management, sales transactions, and billing processes for a shoe wholesaler named SpeedzWear. This project focuses on reading product data from a text file, managing stock updates in real time, calculating conditional discounts, and generating unique notes for both the customers and vendors. It stimulates real wholesale environment by mixing quantity and origin basis discounts for domestic products. This application allows users to view stock, sell products to customers, restock products from suppliers, and generate transaction notes. The product records are stored in a structured text file which will act as this project's storage system, also known as database (Guttag, 2016).

1.2. Aim

The main aim of this project is to develop a Python and file grounded wholesale management system that automates stock tracking, customer sales, restocking procedures and billing for the shoe wholesaler named SpeedzWear.

1.3. Objectives

- First, to create a Python program that reads shoes data from a text file and store it in proper data structures.
- Second, displaying available stock information in readable format for administrators and support restocking of inventory and updating the stock file after every sale or restock.

- Third, allow sales transactions for customers, apply conditional discounts along the process and generate invoices for each transaction.
- Fourth, to ensure error handling and data validation throughout the system and to maintain modularity using proper functions and proper coding practices (Zelle, 2017).

1.4. Technologies Used

- Python IDE IDLE is used to create the development part of the project. IDLE also known as Integrated Development and Learning Environment is a python built-in editor and interpreter. It provides an interactive shell where python commands can be executed where it highlights syntax, errors and program execution. In this program it is used to run a shoe inventory management system where it allows for selling, restocking and invoice generation.
- Notepad is used as the database for the whole system. It is used for shoes.txt to update shoes details and stock. The data is stored in a .txt file to side with the objective of learning file handling and storage based on text.
- Word is used to write documents for various types of occasions. It is a Microsoft product. In this project it is used to write the report for the whole program.

2. Discussion and Analysis

2.1. Algorithm

Step 1: START

Step 2: READ inventory from file using `StockReader.load_stock("shoes.txt")`

Step 3: DISPLAY main menu:

1. Display Stock
2. Sell (Customer)
3. Restock (Vendor)
4. Exit

Step 4: INPUT user choice (as string: "1", "2", "3", "4")

Step 5:

IF choice == "1" THEN → GOTO Step 6
ELSE IF choice == "2" THEN → GOTO Step 10
ELSE IF choice == "3" THEN → GOTO Step 18
ELSE IF choice == "4" THEN → GOTO Step 26
ELSE → GOTO Step 3

Step 6: CALL `WholesaleOperations.show_stock(items)`

Step 7: GOTO Step 3

Step 10: INPUT Customer Name

→ via `input("Customer name: ")`

Step 11: INPUT Product ID and Quantity as comma-separated input

→ inside loop `input("Enter as 'No,Qty' or 'done' ")`

Step 12:

IF done is entered and no purchases made
→ DISPLAY "No products selected. Invoice not generated."
→ GOTO Step 3
ELSE → GOTO Step 13

Step 13: PARSE and validate Quantity

Step 14:

IF Product ID not in inventory
→ append "Invalid Product ID" to errors

Step 15:

IF Quantity ≤ 0
→ append "Invalid Quantity" to errors

Step 16:

IF Quantity > available stock
→ append "Not Enough Stock" to errors

Step 17:

IF any errors exist
→ DISPLAY error messages
→ GOTO Step 3
ELSE
→ CALCULATE discount using `_discount_for_line()`
→ ADD to cart, deduct stock
→ ASK: "More items? (yes/no)"
 IF yes → GOTO Step 11
 IF no → GENERATE invoice using `StockWriter.write_customer_invoice()`
→ GOTO Step 3

Step 18: INPUT Supplier Name

Step 19: INPUT Product ID and Quantity

→ inside loop: `input("Enter as 'No,Qty': ")`

Step 20:

IF done is entered and no restocks made
→ DISPLAY "No products selected. Restock invoice not generated."
→ GOTO Step 3
ELSE → GOTO Step 21

Step 21: Validate Quantity

Step 22:

IF Product ID not in inventory
→ append "Invalid Product ID" to errors

Step 23:

IF Quantity ≤ 0
→ append "Invalid Quantity" to errors

Step 24:

IF any errors exist
→ DISPLAY error messages
→ GOTO Step 3
ELSE
→ ADD quantity to stock
→ GENERATE vendor restock invoice using StockWriter.write_vendor_note()
→ GOTO Step 3

Step 26:

CALL StockWriter.save_stock("shoes.txt", items)
DISPLAY "Saved. Goodbye."

Step 27: END

2.2. Flowchart

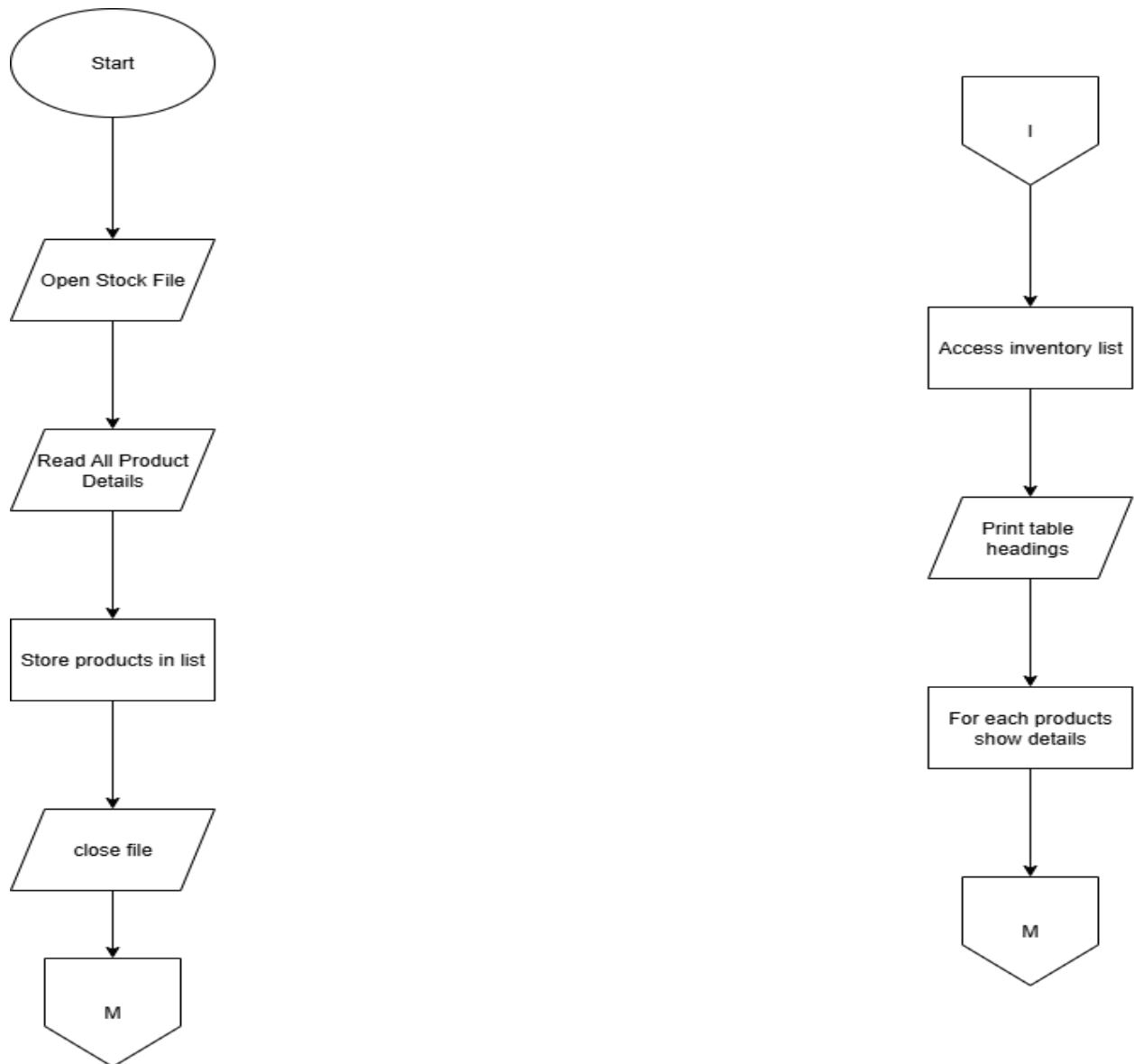


Figure 1: Flowchart

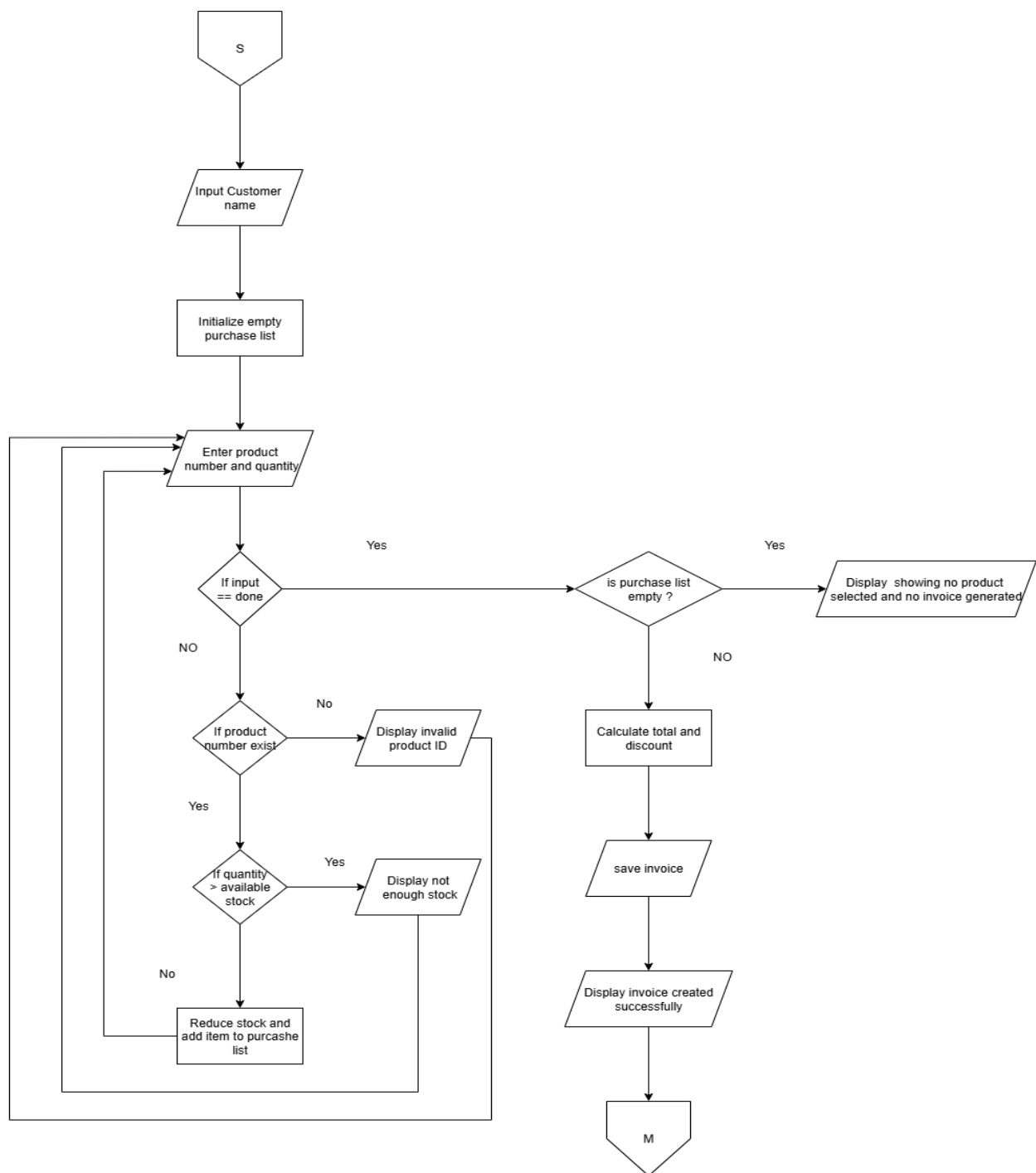


Figure 2: Flowchart



Figure 3: Flowchart

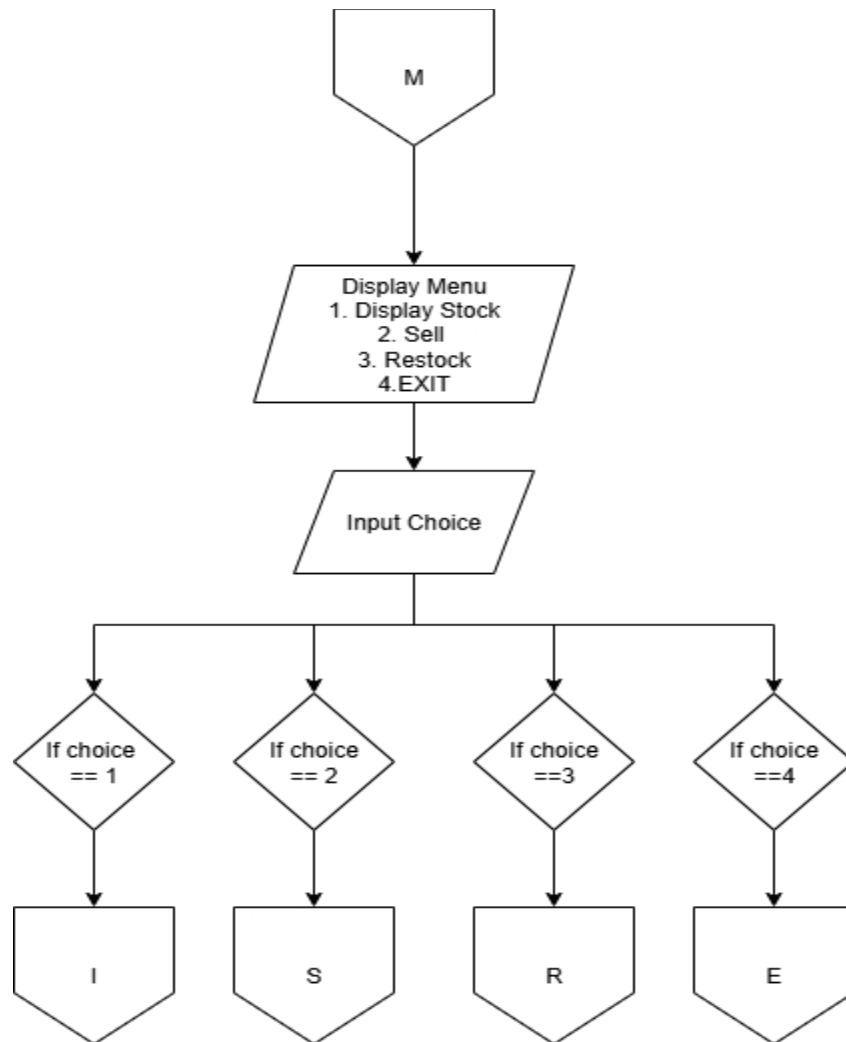


Figure 4: Flowchart

2.3. Pseudocode

2.3.1. read.py

algorithm loadstock

input file_path

output item_list

open file in read mode

initialize item_list as empty list

for each line in file do

split line by ","

create dictionary with no, type, brand, qty, rate, origin

append dictionary to item_list

end for

return item_list

end loadstock

2.3.2. operations.py

algorithm get_discount

input origin, qty

output discount

if qty > 10 then

if origin == "domestic" then

return 0.12

```
    else
        return 0.05
    end if
else
    return 0.0
end if
end get_discount
```

```
algorithm show_stock
input items
if items is empty then
    display "Stock is empty"
else
    display table header
    for each item in items do
        display formatted item row
    end for
end if
end show_stock
```

```
algorithm sell_flow
input items
output updated items
```


ask for customer name

if empty then

display warning and return

initialize empty cart

while true do

input "No,Qty" or 'done'

if input is 'done' then break

validate input

find matching item in items

if not found or quantity too high then

show error

else

calculate discount and line total

add to cart

deduct from stock

end if

end while

if cart is empty then

display warning

else

calculate totals

generate invoice

display confirmation

end if

end sell_flow

algorithm restock_flow

input items

output updated items and vendor restock note

ask for vendor name

if vendor name is empty then return

initialize restocked_items as empty list

while true do

input "No,Qty" or 'done'

if input is 'done' then break

validate input format and values

find item by number in items list

if item found then

add quantity to item stock

calculate line total = qty × rate

append item info and line total to restocked_items

display updated quantity

else

show error

end if

end while

if restocked_items is not empty then

generate vendor restock note using restocked_items

display confirmation

else

display "no valid restocks"

end if

end restock_flow

2.3.3. write.py

algorithm savestock

input file_path, item_list

open file in write mode

for each item in item_list do

write item data as comma-separated string

end for

end savestock

algorithm write_customer_invoice

input customer_name, cart, grand_total, discount

output invoice_filename

generate unique filename with timestamp

open file with invoice_filename in write mode

write header and metadata

for each item in cart do

write item details to file

end for

write total discount and grand total

return invoice_filename

end write_customer_invoice

algorithm write_vendor_invoice

input vendor_name, item, quantity, total_cost

output invoice_filename

generate unique filename with timestamp

open file with invoice_filename in write mode

write vendor name, date, item details

write quantity, cost, and total amount

return invoice_filename

end write_vendor_invoice

2.3.4. main.py

algorithm mainmenu

read inventory from "shoes.txt"

while true do

```
display "1. Display Stock"

display "2. Sell to Customer"

display "3. Restock from Vendor"

display "4. Save and Exit"

input choice

if choice == "1" then

    call show_stock(items)

else if choice == "2" then

    call sell_flow(items)

else if choice == "3" then

    call restock_flow(items)

else if choice == "4" then

    call save_stock("shoes.txt", items)

    display "Saved. Goodbye."

    break

else

    display "Invalid option"

end if

end while

end mainmenu
```

2.4. Data Structure

2.4.1. Integer

Integer (int) means whole numbers but not decimals. It is used for shoe ids and quantities (Guttag, 2016).

```
item = {"no": 3, "type": "Lite Racer", "brand": "Adidas", "qty": 200, "rate": 7000.0, "origin": "international"}
print(item["no"])    # int = 3 (shoe number)
print(item["qty"])   # int = 200 (stock quantity)
```

Figure 5: Integer

2.4.2. Float

Float (float) means numbers with decimals. It is used for price, discount calculations and line totals (Guttag, 2016).

```
item = {"rate": 12000.0, "qty": 2}
total = item["rate"] * item["qty"]    # float calculation
print(total)    # 24000.0
```

Figure 6: Float

2.4.3. String

String (str) means text inside “ ”. It is used for shoe brands, type, origin, and customer/vendor names (Guttag, 2016).

```
item = {"type": "Loafer Light", "brand": "GoldStar", "origin": "domestic"}
print(item["brand"])    # "GoldStar"
print(item["origin"])   # "domestic"
```

Figure 7: String

2.4.4. Boolean

Boolean (bool) is only TRUE or FALSE. It is used to check for availability of something or for discount eligibility (Guttag, 2016).

```
qty = 12
origin = "domestic"
has_discount = (qty > 10 and origin == "domestic") # boolean expression
print(has_discount) # True
```

Figure 8: Boolean

2.4.5. List

Ordered Collection of items. It is used to store inventory items or all products in customers' buy list (Downey, 2015).

```
inventory = [
    {"no": 1, "type": "Loafer Light", "brand": "GoldStar", "qty": 200, "rate": 1000.0, "origin": "domestic"},
    {"no": 2, "type": "Inigo 732", "brand": "Caliber", "qty": 100, "rate": 2800.0, "origin": "domestic"}
]
print(inventory[0]["brand"]) # "GoldStar"
```

Figure 9: List

2.4.6. Tuple

Ordered collection like list but immutable. It is used if fixed value is required (Downey, 2015).

```
origins = ("domestic", "international") # tuple
print(origins[0]) # "domestic"
```

Figure 10: Tuple

2.4.7. Set

Unordered Collection of unique items. It is used to check unique brands or types without repetition (Downey, 2015).

```
brands = {"Nike", "Adidas", "Nike", "GoldStar"}  
print(brands)  # {"Nike", "Adidas", "GoldStar"}  # duplicate removed
```

Figure 11: Set

2.4.8. Dictionary

Dictionary (dic) works in key-value pair. It is used to represent each shoe item or restock action (Guttag, 2016).

```
item = {"no": 4, "type": "Air Max Dn8", "brand": "Nike", "qty": 150, "rate": 12000.0, "origin": "international"}  
print(item["type"])  # "Air Max Dn8"  
print(item["rate"])  # 12000.0
```

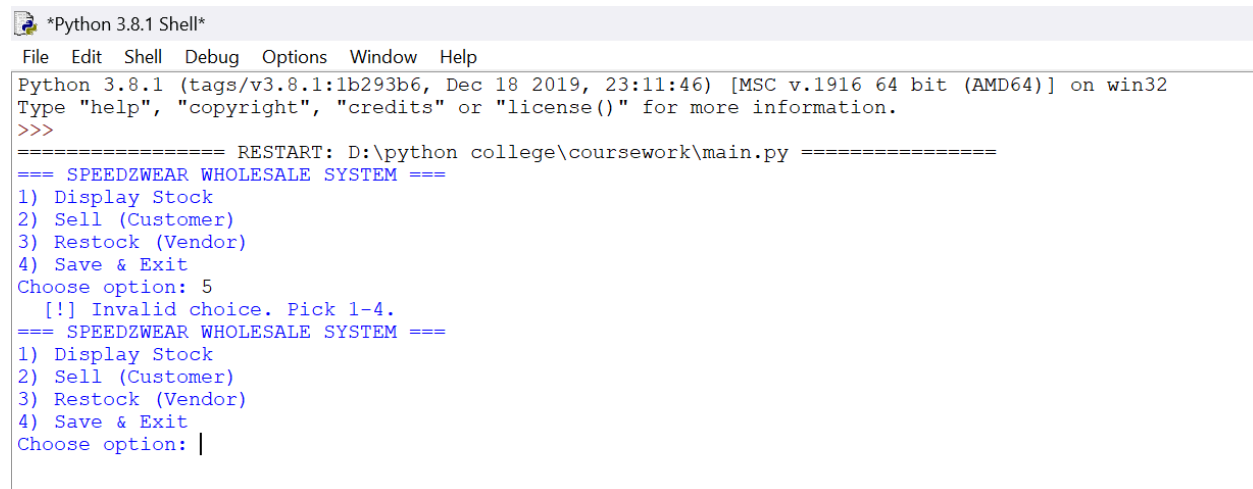
Figure 12: Dictionary

3. Testing

3.1. Test 1

Objective	Showing implementation of try, except in the python run shell.
Action	Providing invalid input first.
Expected result	When invalid input is entered the code will not crash and will display an error message.
Actual result	When invalid input was entered the code did not crash and displayed an error message.
Conclusion	The test was successful.

Table 1: Test 1



```

Python 3.8.1 Shell
File Edit Shell Debug Options Window Help
Python 3.8.1 (tags/v3.8.1:1b293b6, Dec 18 2019, 23:11:46) [MSC v.1916 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: D:\python college\coursework\main.py =====
=== SPEEDZWEAR WHOLESALE SYSTEM ===
1) Display Stock
2) Sell (Customer)
3) Restock (Vendor)
4) Save & Exit
Choose option: 5
[!] Invalid choice. Pick 1-4.
=== SPEEDZWEAR WHOLESALE SYSTEM ===
1) Display Stock
2) Sell (Customer)
3) Restock (Vendor)
4) Save & Exit
Choose option: |

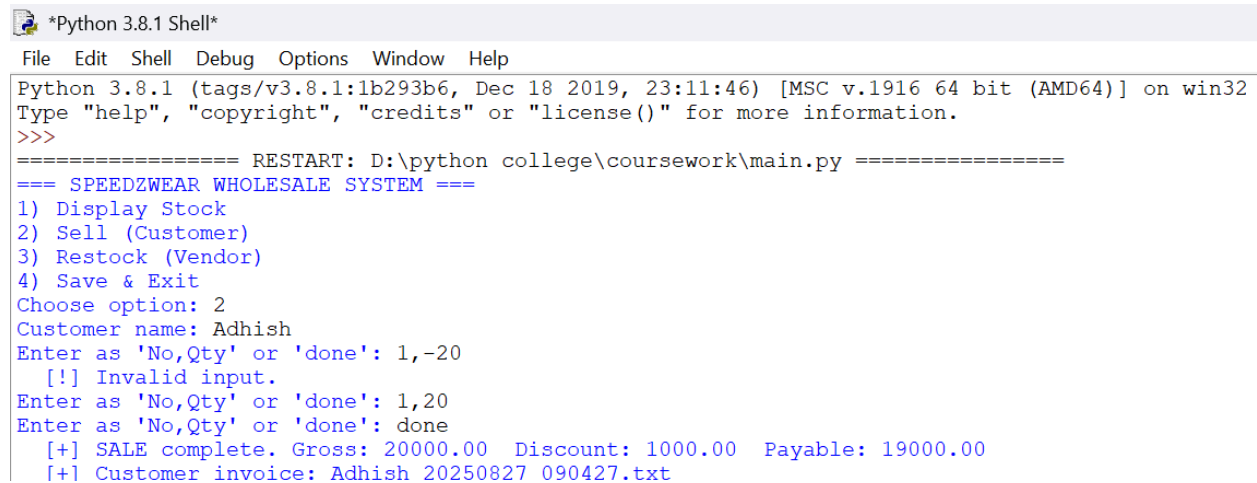
```

Figure 13: Test 1

3.2.1 Test 2.1

Objective	Checking the functionality of sales modules in the python program.
Action	Providing negative value as input to check if it takes negative number as an input.
Expected result	When invalid input is entered the code will not abruptly end and display an error message
Actual result	When invalid input was entered the code did not crash and displayed an error message
Conclusion	The test was successful

Table 2: Test 2.1



```

*Python 3.8.1 Shell*
File Edit Shell Debug Options Window Help
Python 3.8.1 (tags/v3.8.1:1b293b6, Dec 18 2019, 23:11:46) [MSC v.1916 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: D:\python college\coursework\main.py =====
=== SPEEDZWEAR WHOLESALE SYSTEM ===
1) Display Stock
2) Sell (Customer)
3) Restock (Vendor)
4) Save & Exit
Choose option: 2
Customer name: Adhish
Enter as 'No,Qty' or 'done': 1,-20
[!] Invalid input.
Enter as 'No,Qty' or 'done': 1,20
Enter as 'No,Qty' or 'done': done
[+] SALE complete. Gross: 20000.00 Discount: 1000.00 Payable: 19000.00
[+] Customer invoice: Adhish 20250827 090427.txt

```

Figure 14: Test 2.1

3.2.2. Test 2.2

Objective	Checking the functionality of sales in the python program.
Action	Providing no value as input.
Expected result	When invalid input is entered the code will not stop and display an error message.
Actual result	When invalid input was entered the code did not stop and displayed an error message.
Conclusion	The test was successful.

Table 3: Test 2.2

```

=== SPEEDZWEAR WHOLESALE SYSTEM ===
1) Display Stock
2) Sell (Customer)
3) Restock (Vendor)
4) Save & Exit
Choose option: 2
Customer name: Adhish
Enter as 'No,Qty' or 'done': 1,
[!] Invalid input.
Enter as 'No,Qty' or 'done':
[!] Format: No,Qty
Enter as 'No,Qty' or 'done': 2,3
Enter as 'No,Qty' or 'done': done
[+] SALE complete. Gross: 8400.00 Discount: 0.00 Payable: 8400.00
[+] Customer invoice: Adhish_20250827_090740.txt
=== SPEEDZWEAR WHOLESALE SYSTEM ===
1) Display Stock
2) Sell (Customer)
3) Restock (Vendor)
4) Save & Exit
Choose option: |

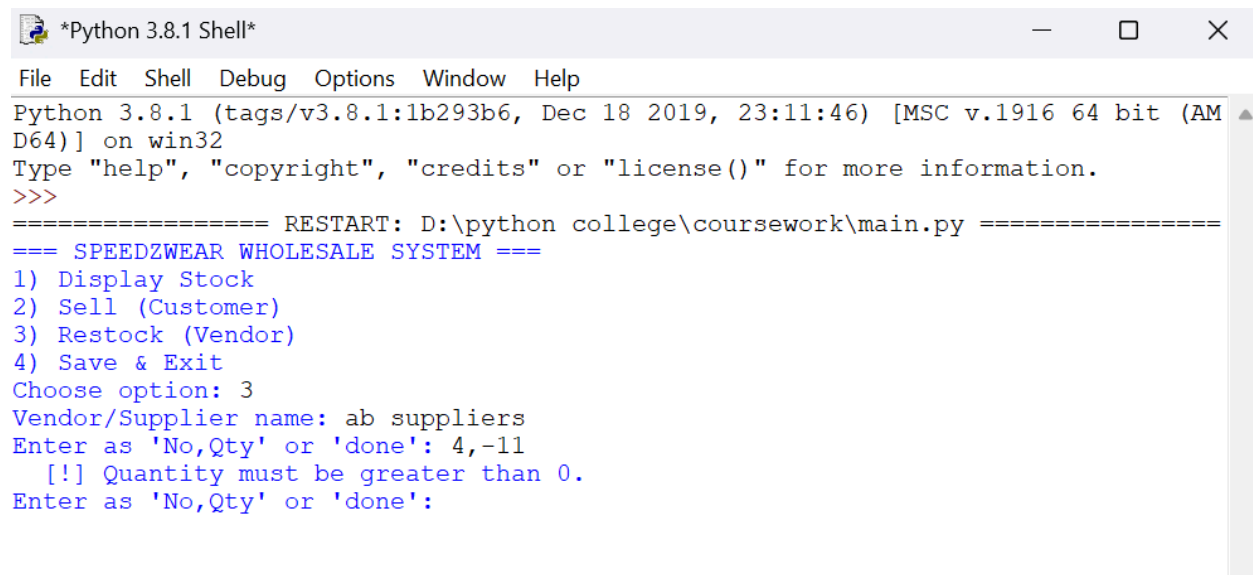
```

Figure 15: Test 2.2

3.2.3. Test 2.3

Objective	Checking the functionality of restock in the python program.
Action	Providing negative value as input.
Expected result	When invalid input is entered the code will not stop and display an error message.
Actual result	When invalid input was entered the code did not stop and displayed an error message.
Conclusion	The test was successful.

Table 4: Test 2.3



```

Python 3.8.1 Shell*
File Edit Shell Debug Options Window Help
Python 3.8.1 (tags/v3.8.1:1b293b6, Dec 18 2019, 23:11:46) [MSC v.1916 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: D:\python college\coursework\main.py =====
=== SPEEDZWEAR WHOLESALE SYSTEM ===
1) Display Stock
2) Sell (Customer)
3) Restock (Vendor)
4) Save & Exit
Choose option: 3
Vendor/Supplier name: ab suppliers
Enter as 'No,Qty' or 'done': 4,-11
    [!] Quantity must be greater than 0.
Enter as 'No,Qty' or 'done':

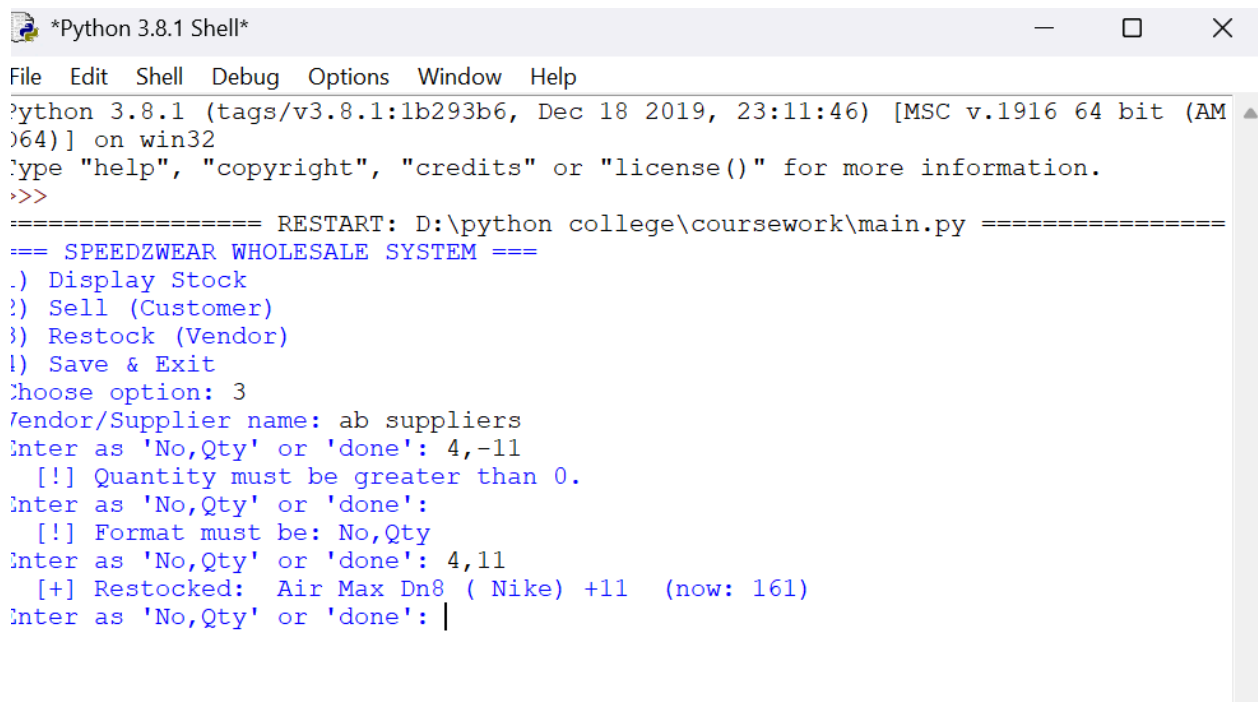
```

Figure 16: Test 2.3

3.2.4. Test 2.4

Objective	Checking the functionality of restock in the program.
Action	Providing no value as input.
Expected result	When invalid input is entered the code should not crash and display an error message.
Actual result	When invalid input was entered the code did not crash and displayed an error message.
Conclusion	The test was successful.

Table 5: Test 2.4



```

Python 3.8.1 (tags/v3.8.1:1b293b6, Dec 18 2019, 23:11:46) [MSC v.1916 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: D:\python college\coursework\main.py =====
=== SPEEDZWEAR WHOLESALE SYSTEM ===
.) Display Stock
?) Sell (Customer)
3) Restock (Vendor)
4) Save & Exit
Choose option: 3
Vendor/Supplier name: ab suppliers
Enter as 'No,Qty' or 'done': 4,-11
[!] Quantity must be greater than 0.
Enter as 'No,Qty' or 'done':
[!] Format must be: No,Qty
Enter as 'No,Qty' or 'done': 4,11
[+] Restocked: Air Max Dn8 ( Nike) +11 (now: 161)
Enter as 'No,Qty' or 'done': |

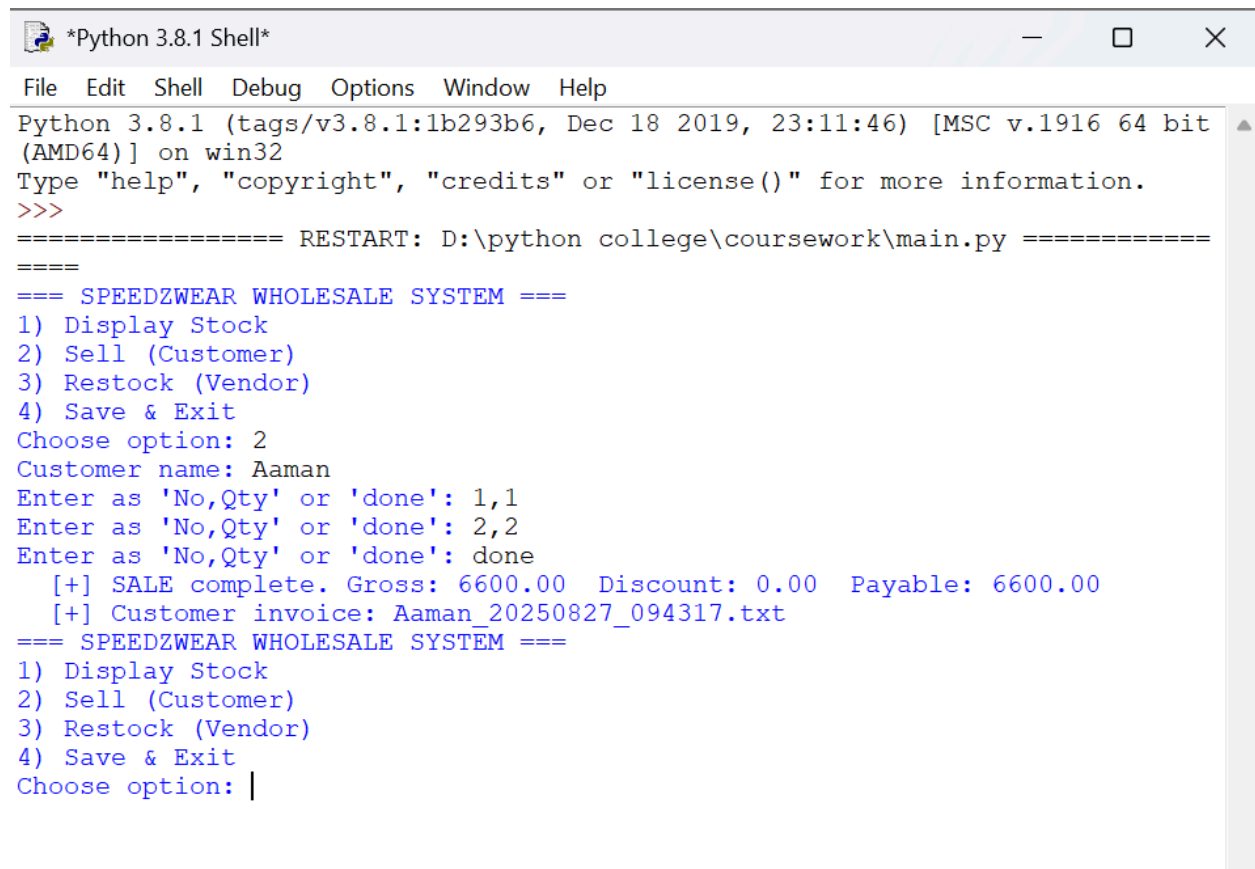
```

Figure 17: Test 2.4

3.3. Test 3

Objective	Checking file generation of sold products.
Action	Selling multiple shoes.
Expected result	The invoice should be generated for the sold products. Invoice should have unique name.
Actual result	The invoice was generated for the sold products. Invoice had unique name.
Conclusion	The test was successful.

Table 6: Test 3



```

Python 3.8.1 Shell
File Edit Shell Debug Options Window Help
Python 3.8.1 (tags/v3.8.1:1b293b6, Dec 18 2019, 23:11:46) [MSC v.1916 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: D:\python college\coursework\main.py =====
=====
=== SPEEDZWEAR WHOLESALE SYSTEM ===
1) Display Stock
2) Sell (Customer)
3) Restock (Vendor)
4) Save & Exit
Choose option: 2
Customer name: Aaman
Enter as 'No,Qty' or 'done': 1,1
Enter as 'No,Qty' or 'done': 2,2
Enter as 'No,Qty' or 'done': done
[+] SALE complete. Gross: 6600.00 Discount: 0.00 Payable: 6600.00
[+] Customer invoice: Aaman_20250827_094317.txt
=== SPEEDZWEAR WHOLESALE SYSTEM ===
1) Display Stock
2) Sell (Customer)
3) Restock (Vendor)
4) Save & Exit
Choose option: |

```

Figure 18: Test 3

```

SPEEDZWEAR - CUSTOMER INVOICE
-----
Customer: Aaman
Date: 2025-08-27 09:43:17.813604

No Type                Brand      Qty    Rate    Disc%   Line Total
-----
1 Loafer Light         GoldStar  1    1000.00   0.0%    1000.00
2 Inigo 732            Caliber   2    2800.00   0.0%    5600.00
-----
Total Discount Applied: 0.00
Grand Total (NPR): 6600.00
-----

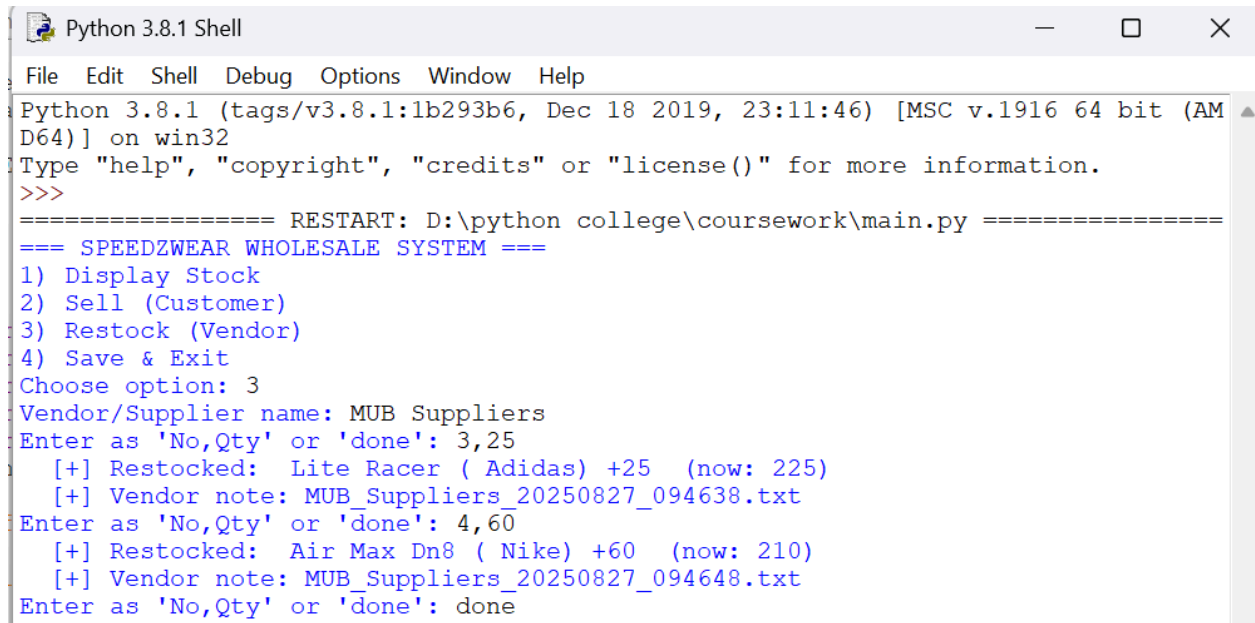
```

Figure 19: Test 3

3.4 Test 4

Objective	Checking file generation of restock products.
Action	Restocking multiple shoes.
Expected result	The invoice should be generated for the sold products. Invoice should have unique name.
Actual result	The invoice was generated for the sold products. Invoice had unique name.
Conclusion	The test was successful.

Table 7: Test 4

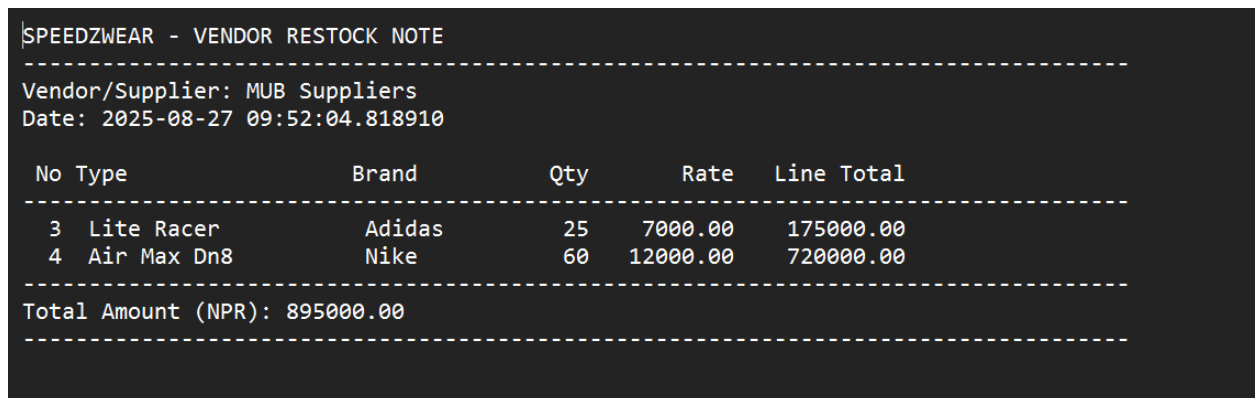


```

Python 3.8.1 Shell
File Edit Shell Debug Options Window Help
Python 3.8.1 (tags/v3.8.1:1b293b6, Dec 18 2019, 23:11:46) [MSC v.1916 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: D:\python college\coursework\main.py =====
=== SPEEDZWEAR WHOLESALE SYSTEM ===
1) Display Stock
2) Sell (Customer)
3) Restock (Vendor)
4) Save & Exit
Choose option: 3
Vendor/Supplier name: MUB Suppliers
Enter as 'No,Qty' or 'done': 3,25
    [+] Restocked: Lite Racer ( Adidas) +25 (now: 225)
    [+] Vendor note: MUB_Suppliers_20250827_094638.txt
Enter as 'No,Qty' or 'done': 4,60
    [+] Restocked: Air Max Dn8 ( Nike) +60 (now: 210)
    [+] Vendor note: MUB_Suppliers_20250827_094648.txt
Enter as 'No,Qty' or 'done': done

```

Figure 20: Test 4



```

SPEEDZWEAR - VENDOR RESTOCK NOTE
-----
Vendor/Supplier: MUB Suppliers
Date: 2025-08-27 09:52:04.818910

No Type                Brand      Qty    Rate    Line Total
-----
 3 Lite Racer           Adidas     25    7000.00  175000.00
 4 Air Max Dn8          Nike      60   12000.00  720000.00
-----
Total Amount (NPR): 895000.00
-----

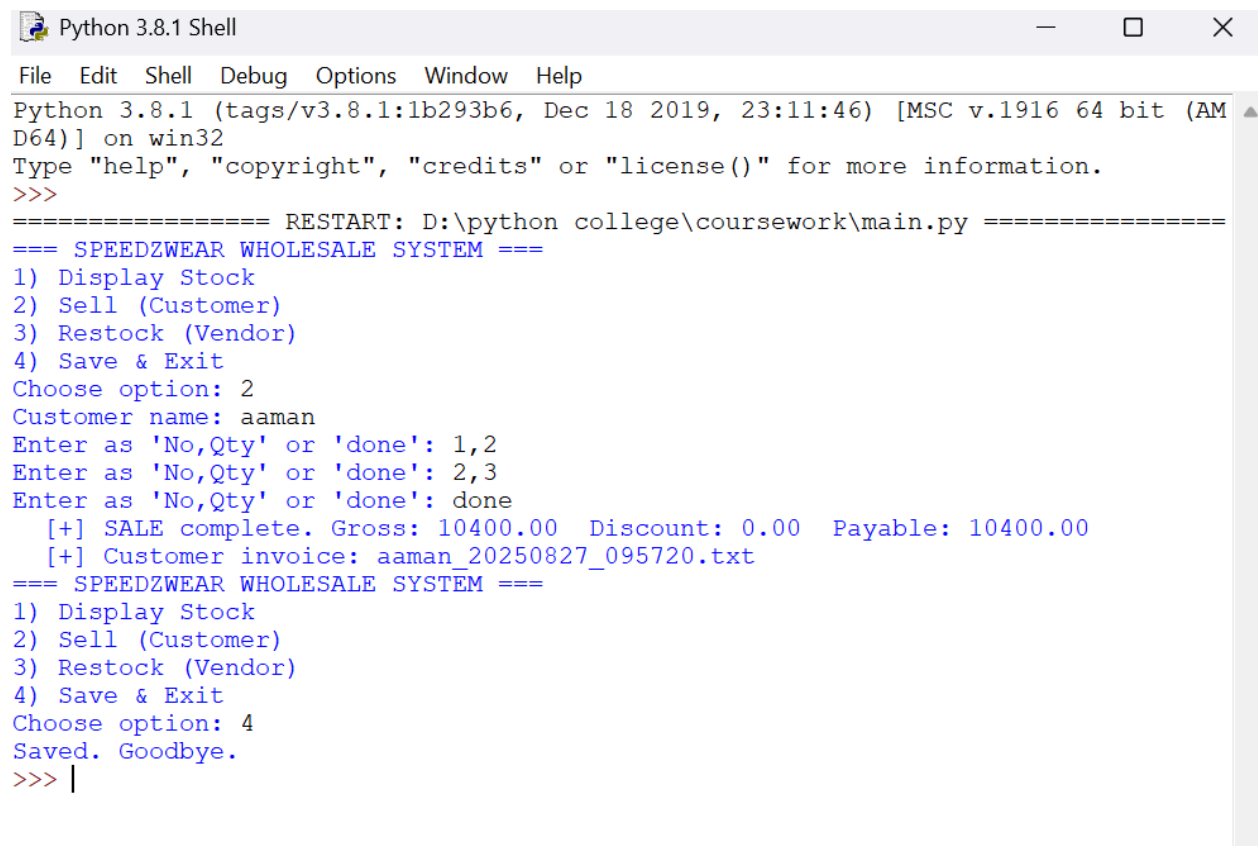
```

Figure 21: Test 4

3.5.1. Test 5.1

Objective	Checking if stock file is updated for sales of products.
Action	Checking stock file after selling shoes.
Expected result	Shoe quantity should decrease as the number of shoes purchased.
Actual result	Shoe quantity decreased as the number of shoes purchased.
Conclusion	The test was successful.

Table 8: Test 5.1



```

Python 3.8.1 Shell
File Edit Shell Debug Options Window Help
Python 3.8.1 (tags/v3.8.1:1b293b6, Dec 18 2019, 23:11:46) [MSC v.1916 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: D:\python college\coursework\main.py =====
=== SPEEDZWEAR WHOLESALE SYSTEM ===
1) Display Stock
2) Sell (Customer)
3) Restock (Vendor)
4) Save & Exit
Choose option: 2
Customer name: aaman
Enter as 'No,Qty' or 'done': 1,2
Enter as 'No,Qty' or 'done': 2,3
Enter as 'No,Qty' or 'done': done
[+] SALE complete. Gross: 10400.00 Discount: 0.00 Payable: 10400.00
[+] Customer invoice: aaman_20250827_095720.txt
=== SPEEDZWEAR WHOLESALE SYSTEM ===
1) Display Stock
2) Sell (Customer)
3) Restock (Vendor)
4) Save & Exit
Choose option: 4
Saved. Goodbye.
>>> |

```

Figure 22: Test 5.1

```

1, Loafer Light, GoldStar, 200, 1000, domestic
2, Inigo 732, Caliber, 100, 2800, domestic
3, Lite Racer, Adidas, 200, 7000, international
4, Air Max Dn8, Nike, 150, 12000, international

```

Figure 23: Test 5.1 Before

```

1, Loafer Light, GoldStar,198,1000.0, domestic
2, Inigo 732, Caliber,97,2800.0, domestic
3, Lite Racer, Adidas,200,7000.0, international
4, Air Max Dn8, Nike,150,12000.0, international

```

Figure 24: Test 5.1 After

3.5.2. Test 5.2

Objective	Checking if stock file is updated for restock of products.
Action	Checking stock file after restocking shoes.
Expected result	Shoe quantity should increase as the number of shoes restocked.
Actual result	Shoe quantity increased as the number of shoes purchased.
Conclusion	The test was successful.

Table 9: Test 5.2

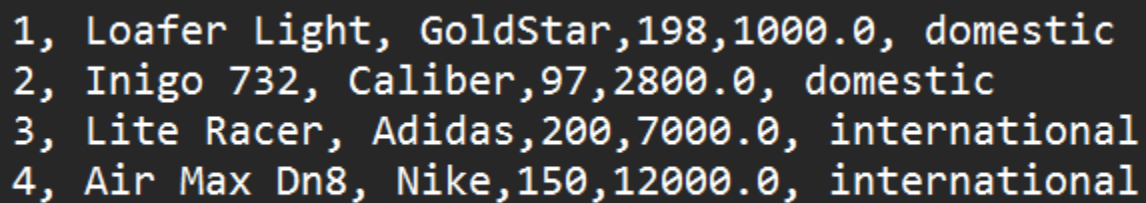


```

Python 3.8.1 Shell
File Edit Shell Debug Options Window Help
Python 3.8.1 (tags/v3.8.1:1b293b6, Dec 18 2019, 23:11:46) [MSC v.1916 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: D:\python college\coursework\main.py =====
=== SPEEDZWEAR WHOLESALE SYSTEM ===
1) Display Stock
2) Sell (Customer)
3) Restock (Vendor)
4) Save & Exit
Choose option: 3
Vendor/Supplier name: MUB Suppliers
Enter as 'No,Qty' or 'done': 3,30
[+] Restocked: Lite Racer ( Adidas) +30 (now: 230)
Enter as 'No,Qty' or 'done': 4,60
[+] Restocked: Air Max Dn8 ( Nike) +60 (now: 210)
Enter as 'No,Qty' or 'done': done
[+] Vendor restock note saved as: MUB_Suppliers_20250827_095953.txt
=== SPEEDZWEAR WHOLESALE SYSTEM ===
1) Display Stock
2) Sell (Customer)
3) Restock (Vendor)
4) Save & Exit
Choose option: 4
Saved. Goodbye.
>>>

```

Figure 25: Test 5.2



```

1, Loafer Light, GoldStar,198,1000.0, domestic
2, Inigo 732, Caliber,97,2800.0, domestic
3, Lite Racer, Adidas,200,7000.0, international
4, Air Max Dn8, Nike,150,12000.0, international

```

Figure 26: Test 5.2 Before

```
1, Loafer Light, GoldStar,198,1000.0, domestic
2, Inigo 732, Caliber,97,2800.0, domestic
3, Lite Racer, Adidas,230,7000.0, international
4, Air Max Dn8, Nike,210,12000.0, international
```

Figure 27: Test 5.2 After

5. Conclusion

In this project, I learned how different Python data structures are used in real programs. Integers and floats helped with numbers like quantity and price, strings were used for names and brands, and booleans handled true/false conditions. Lists and dictionaries were the main ways to store and manage the inventory, while tuples and sets showed how to work with fixed values and unique data. By combining all these, I was able to build a system that can read stocks, sell to customers, restock from vendors, and save invoices. This showed me how important data structures are for solving practical problems in inventory and sales management.

6. References

- Downey, A. (2015). Think Python: How to Think Like a Computer Scientist. Green Tea Press.
- Guttag, J. (2016). Introduction to Computation and Programming Using Python. MIT Press.
- Zelle, J. (2017). Python Programming: An Introduction to Computer Science. Franklin, Beedle & Associates.

7. Appendix

```
"""
```

```
read.py
```

Provides utility function to read stock data from a structured text file and convert it into a list of dictionaries containing shoe details.

Each line of the file is parsed and validated to extract:

- Shoe number (int)
- Type (str)
- Brand (str)
- Quantity (int)
- Rate (float)
- Origin (str)

```
"""
```

```
class StockReader:
```

```
    """
```

Reads the stock.txt file and returns the stock data as a list of dictionaries.

Parameters:file_path (str): The path to the stock file.

Returns:list of dict: A list of dictionaries representing each shoe's stock entry.

"""

```
def load_stock(path):  
    items = []  
    try:  
        with open(path, "r") as f:  
            for raw in f:  
                line = raw.replace("\n", "").replace("\r", "")  
                if line == "":  
                    continue  
                parts = line.split(",")  
                if len(parts) != 6:  
                    print(" [!] Skipped malformed line:", line)  
                    continue  
  
                no_s = parts[0]  
                s_type = parts[1]  
                brand = parts[2]  
                qty_s = parts[3]  
                rate_s = parts[4]  
                origin = parts[5].lower()  
  
    try:
```

```
        no = int(no_s)

        if no <= 0: raise ValueError

    except:

        print(" [!] Invalid shoe number, skipping:", line)

        continue

    try:

        qty = int(qty_s)

        if qty < 0: raise ValueError

    except:

        print(" [!] Invalid qty, skipping:", line)

        continue

    try:

        rate = float(rate_s)

        if rate < 0: raise ValueError

    except:

        print(" [!] Invalid rate, skipping:", line)

        continue


    items.append({

        "no": no,

        "type": s_type,

        "brand": brand,

        "qty": qty,
```

```
        "rate": rate,
        "origin": origin
    })

except FileNotFoundError:

    print(" [!] " + path + " not found. Starting with empty stock.")

    return items

"""

operation.py
```

Contains the main operational logic of the SpeedzWear Wholesale System.

Includes functionalities for displaying stock, selling to customers,
and restocking from vendors.

Uses:

- show_stock(): Display current stock in tabular format.
- sell_flow(): Handle customer purchases and discount calculations.
- restock_flow(): Handle vendor restocking and update inventory accordingly.

"""

```
from write import StockWriter
```

```
class WholesaleOperations:
```

"""

Class that encapsulates the main operational workflows for wholesale transactions.

"""

```
def _discount_for_line(origin, qty):
```

```
    """
```

Calculates applicable discount for a line item based on origin and quantity.

Parameters: origin (str): 'domestic' or 'imported'

qty (int): Quantity being purchased

Returns:float: Discount rate (e.g., 0.12 for 12%)

```
    """
```

```
    if qty > 10:
```

```
        if origin == "domestic":
```

```
            return 0.12
```

```
        return 0.05
```

```
    return 0.0
```

```
def show_stock(items):
```

```
    """
```

Prints the current stock in a formatted table.

Parameters: items (list of dict): The current stock list.

```
    """
```

```
    if not items:
```

```

    print("\n[Stock is empty]\n")

    return

print("\n---- CURRENT STOCK ----")

    print("{:>3} {:25} {:15} {:>5} {:>10} {:>15}".format("No", "Type", "Brand", "Qty",
"Rate", "Origin"))

    print("-" * 78)

    for it in items:

        print("{:>3} {:25} {:15} {:>5} {:>10.2f} {:>15}".format(

            it["no"], it["type"], it["brand"], it["qty"], it["rate"], it["origin"]

        ))

    print("-" * 78 + "\n")

```

def sell_flow(items):

"""

Handles the sales process with a customer.

- Prompts for customer name
- Accepts product number and quantity
- Applies conditional discounts
- Updates stock and generates a customer invoice

Parameters: items (list of dict): The current stock list (will be updated).

"""

```
customer = input("Customer name: ")

if customer.replace(" ", "") == "":
    print(" [!] Customer name required.")
    return

cart = []

while True:
    raw = input("Enter as 'No,Qty' or 'done': ")
    if raw.lower().replace(" ", "") == "done":
        break

    parts = raw.replace("\n", "").replace("\r", "").split(",")
    if len(parts) != 2:
        print(" [!] Format: No,Qty")
        continue

    try:
        no = int(parts[0])
        qty = int(parts[1])
        if qty <= 0:
            raise ValueError
    except:
        print(" [!] Invalid input.")
```

```
        continue

    found = next((it for it in items if it["no"] == no), None)

    if not found:

        print(" [!] Shoe not found.")

        continue

    if found["qty"] < qty:

        print(" [!] Not enough stock. Available: " + str(found["qty"]))

        continue

    disc = WholesaleOperations._discount_for_line(found["origin"], qty)

    line_total = (found["rate"] * qty) * (1.0 - disc)

    cart.append({

        "no": found["no"],

        "type": found["type"],

        "brand": found["brand"],

        "qty": qty,

        "rate": found["rate"],

        "disc": disc,

        "line_total": line_total

    })
```

```
found["qty"] -= qty
```

```
if not cart:
```

```
    print(" [!] No items in cart.")
```

```
    return
```

```
gross = sum(item["rate"] * item["qty"] for item in cart)
```

```
discounted = sum(item["line_total"] for item in cart)
```

```
total_discount = gross - discounted
```

```
    fname = StockWriter.write_customer_invoice(customer, cart, discounted,  
total_discount)
```

```
    print(" [+] SALE complete. Gross: {:.2f} Discount: {:.2f} Payable:  
{:.2f}".format(gross, total_discount, discounted))
```

```
    print(" [+] Customer invoice: " + fname)
```

```
def restock_flow(items):
```

```
    """
```

```
    Handles the restocking process with a vendor.
```

```
- Prompts for vendor name
```

```
- Accepts multiple product entries in format 'No,Qty'
```

```
- Updates stock quantities
```

- Generates a vendor invoice for all valid restocks

Parameters: items (list of dict): The current stock list (will be updated).

```
"""
```

```
vendor = input("Vendor/Supplier name: ")
```

```
if vendor.replace(" ", "") == "":
```

```
    print(" [!] Vendor name required.")
```

```
    return
```

```
restocked_items = []
```

```
while True:
```

```
    raw = input("Enter as 'No,Qty' or 'done': ")
```

```
    raw_no_space = raw.replace(" ", "").replace("\n", "").replace("\r", "")
```

```
    if raw_no_space.lower() == "done":
```

```
        break
```

```
    if "," not in raw:
```

```
        print(" [!] Format must be: No,Qty")
```

```
        continue
```

```
    parts = raw.split(",")
```

```
if len(parts) != 2:

    print(" [!] Format must be: No,Qty")

    continue


if parts[0] == "" or parts[1] == "":

    print(" [!] Both No and Qty must be provided.")

    continue


try:

    no = int(parts[0])

    qty = int(parts[1])

    if qty <= 0:

        print(" [!] Quantity must be greater than 0.")

        continue


except:

    print(" [!] No and Qty must be valid integers.")

    continue


found = next((it for it in items if it["no"] == no), None)


if not found:

    print(" [!] Shoe not found.")

    continue
```

```
found["qty"] += qty
```

```
total_cost = qty * found["rate"]
```

```
restocked_items.append({
```

```
    "no": found["no"],
```

```
    "type": found["type"],
```

```
    "brand": found["brand"],
```

```
    "origin": found["origin"],
```

```
    "qty": qty,
```

```
    "rate": found["rate"],
```

```
    "line_total": total_cost
```

```
})
```

```
    print(" [+] Restocked: " + found["type"] + " (" + found["brand"] + ") +" + str(qty) +  
" (now: " + str(found["qty"]) + ")")
```

```
if restocked_items:
```

```
    fname = StockWriter.write_vendor_invoice(vendor, restocked_items)
```

```
    print(" [+] Vendor restock note saved as: " + fname)
```

```
else:
```

```
    print(" [!] No valid restocks to save.")
```


"""

write.py

Handles all writing and saving functionalities:

- Saving updated stock list to file
- Generating customer invoices for purchases
- Generating vendor restock notes for suppliers

Invoices and notes are saved as uniquely named text files using datetime stamps.

"""

from datetime import datetime

class StockWriter:

"""

Provides static methods for writing invoices and saving stock data to files.

"""

def save_stock(stock_path, items):

"""

Writes the updated stock list back to the stock file.

Parameters: stock_path (str): File path to write to.

items (list of dict): Updated stock list to write.

```

"""

with open(stock_path, "w") as f:

    i = 0

    while i < len(items):

        it = items[i]

        line = str(it["no"]) + "," + it["type"] + "," + it["brand"] + "," + str(it["qty"]) + "," +
str(it["rate"]) + "," + it["origin"]

        f.write(line + ("\n" if i < len(items) - 1 else ""))

        i += 1

```

```
def _unique_invoice_name(base_name):
```

```

"""

```

Generates a unique invoice filename using the base name and current timestamp.

Parameters: base_name (str): Customer or vendor name

Returns: str: Unique filename ending in '.txt'

```

"""

```

```
clean = base_name.replace(" ", "_")
```

```
return clean + "_" + datetime.now().strftime("%Y%m%d_%H%M%S") + ".txt"
```

```
def write_customer_invoice(customer_name, cart, grand_total, grand_discount):
```

```

"""

```

Writes a detailed invoice for a customer transaction.

Parameters:

customer_name (str): Name of the customer.

cart (list of dict): Purchased items.

grand_total (float): Final total after discounts.

grand_discount (float): Total discount applied.

Returns:

str: File name of the generated invoice.

"""

```
filename = StockWriter._unique_invoice_name(customer_name)
```

```
with open(filename, "w") as f:
```

```
    f.write("SPEEDZWEAR - CUSTOMER INVOICE\n")
```

```
    f.write("-" * 84 + "\n")
```

```
    f.write("Customer: " + customer_name + "\n")
```

```
    f.write("Date: " + str(datetime.now()) + "\n\n")
```

```
    f.write("{:>3} {:20} {:12} {:>5} {:>10} {:>7} {:>12}\n".format("No", "Type", "Brand",  
"Qty", "Rate", "Disc%", "Line Total"))
```

```
    f.write("-" * 84 + "\n")
```

```
    i = 0
```

```
    while i < len(cart):
```

```
        line = cart[i]
```

```

        f.write("{:>3} {:20} {:12} {:>5} {:>10.2f} {:>6.1f}% {:>12.2f}\n".format(
            line["no"], line["type"], line["brand"], line["qty"],
            line["rate"], line["disc"] * 100, line["line_total"]
        ))

        i += 1

    f.write("-" * 84 + "\n")

    f.write("Total Discount Applied: " + ("%2f" % grand_discount) + "\n")

    f.write("Grand Total (NPR): " + ("%2f" % grand_total) + "\n")

    f.write("-" * 84 + "\n")

    return filename

```

```
def write_vendor_invoice(vendor_name, restock_list):
```

```
    """
```

Writes a detailed restock note for a vendor with multiple products.

Parameters:

vendor_name (str): Vendor/Supplier name.

restock_list (list of dict): List of restocked items with line totals.

Returns:

str: File name of the generated vendor invoice.

```
    """
```

```
    filename = StockWriter._unique_invoice_name(vendor_name)
```

with open(filename, "w") as f:

```

    f.write("SPEEDZWEAR - VENDOR RESTOCK NOTE\n")

    f.write("-" * 84 + "\n")

    f.write("Vendor/Supplier: " + vendor_name + "\n")

    f.write("Date: " + str(datetime.now()) + "\n\n")

    f.write("{:>3} {:20} {:12} {:>5} {:>10} {:>12}\n".format("No", "Type", "Brand", "Qty",
"Rate", "Line Total"))

    f.write("-" * 84 + "\n")

    total_cost = 0

    for item in restock_list:

        f.write("{:>3} {:20} {:12} {:>5} {:>10.2f} {:>12.2f}\n".format(

            item["no"], item["type"], item["brand"], item["qty"],

            item["rate"], item["line_total"]

        ))

        total_cost += item["line_total"]

    f.write("-" * 84 + "\n")

    f.write("Total Amount (NPR): " + ("%0.2f" % total_cost) + "\n")

    f.write("-" * 84 + "\n")

    return filename

```

"""

main.py

This is the controller script for the SpeedzWear Wholesale Shoe Management System. It handles user interaction via a command-line menu interface and delegates stock operations such as viewing inventory, selling to customers, restocking from vendors, and saving stock updates to separate modules.

Modules Used:

- read.py: For reading the initial stock data from a file
- operation.py: For handling business logic of selling and restocking
- write.py: For generating invoices and saving stock files

"""

from read import StockReader

from write import StockWriter

from operations import WholesaleOperations

STOCK_FILE = "shoes.txt"

def main():

"""

Entry point of the application. Provides the main menu loop where users

can choose to display stock, sell products, restock products, or exit.

Actions:

1. Show Stock – Displays current stock table.
2. Sell Product – Starts customer purchase flow.
3. Restock Product – Starts vendor restock flow.
4. Save and Exit – Writes updated stock to file and exits.

"""

```
items = StockReader.load_stock(STOCK_FILE)
```

```
while True:
```

```
    print("=== SPEEDZWEAR WHOLESALE SYSTEM ===")
```

```
    print("1) Display Stock")
```

```
    print("2) Sell (Customer)")
```

```
    print("3) Restock (Vendor)")
```

```
    print("4) Save & Exit")
```

```
    choice = input("Choose option: ").replace("\n","").replace("\r","")
```

```
    if choice == "1":
```

```
        WholesaleOperations.show_stock(items)
```

```
    elif choice == "2":
```

```
        WholesaleOperations.sell_flow(items)
```

```
    elif choice == "3":
```

```
    WholesaleOperations.restock_flow(items)

elif choice == "4":

    StockWriter.save_stock(STOCK_FILE, items)

    print("Saved. Goodbye.")

    break

else:

    print(" [!] Invalid choice. Pick 1-4.")

if __name__ == "__main__":

    main()
```